

Versatile Cross-platform Compilation Toolchain for Schrödinger-style Quantum Circuit Simulation

Yuncheng Lu, Shuang Liang, Hongxiang Fan, Ce Guo, Wayne Luk, Paul H. J. Kelly
Department of Computing, Imperial College London, London, UK
Email: {yuncheng.lu19, shuang.liang, hongxiang.fan, c.guo, w.luk, p.kelly}@imperial.ac.uk

Abstract—While existing quantum hardware resources have limited availability and reliability, there is a growing demand for exploring and verifying quantum algorithms. Efficient classical simulators for high-performance quantum simulation are critical to meeting this demand. However, due to the vastly varied characteristics of classical hardware, implementing hardware-specific optimizations for different hardware platforms is challenging. To address such needs, we propose CAST (Cross-platform Adaptive Schrödinger-style Simulation Toolchain), a novel compilation toolchain with cross-platform (CPU and Nvidia GPU) optimization and high-performance backend supports. CAST exploits a novel sparsity-aware gate fusion algorithm that automatically selects the best fusion strategy and backend configuration for targeted hardware platforms. CAST also aims to offer versatile and high-performance backend for different hardware platforms. To this end, CAST provides an LLVM IR-based vectorization optimization for various CPU architectures and instruction sets, and a PTX-based code generator for Nvidia GPU support. We benchmark CAST against IBM Qiskit, Google QSimCirq, Nvidia cuQuantum backend, and other high-performance simulators. On various 32-qubit CPU-based benchmarks, CAST achieves up to 8.03x speedup than Qiskit. On various 30-qubit GPU-based benchmarks, CAST achieves up to 39.3x speedup than Nvidia cuQuantum backend.

I. INTRODUCTION

Significant research efforts have focused on developing new quantum algorithms under the assumption of reliable quantum computation [1], [2]. Nevertheless, verifying these algorithms and demonstrating their performance at scale is challenging. Therefore, there is a growing demand for simulators based on classical computers as alternatives to quantum hardware. High-performance and reliable classical simulators play a vital role in identifying the potential of quantum computers, particularly in scenarios where quantum computers claim supremacy in tackling problems that are intractable for classical systems [3], [4], [5], [6]. The performance of simulations also becomes crucial for various applications. For instance, Variational Quantum Algorithms (VQA) [7] often require optimizing parametrized quantum circuits by simulating thousands of runs on the same circuit structure. Similarly, Hamiltonian Evolution Simulation relies on the decomposition of the time-evolution operator [8], which induces a significant circuit depth.

However, simulating the quantum world using classical computers has been recognized as a challenging problem. To perfectly mimic the physics of the quantum world, it requires either exponentially increasing memory or time with respect to system size [9]. Among current simulation approaches, Schrödinger style simulations are often used to facilitate the comprehensive verification and evaluation of quantum algorithms. These methods require exponential memory to keep track of all quantum state information and are capable of handling all kinds of quantum operations.

Although various Schrödinger style simulators have been introduced by both industry [10], [11], [12] and academia [13], [14], [15], they are suffering two key issues in obtaining accessible and high-performant simulations. First, the absence of advanced and versatile simulation-oriented gate rewriting and fusion optimization

hampers their scalability for large quantum circuits. Second, existing simulation tools do not offer comprehensive cross-platform backend support, and there is an urgent need for an easy-to-use cross-platform solution to accommodate the diverse needs of users with varying hardware resources. To address these challenges, we propose an adaptive and cross-platform (with CPU and Nvidia GPU support) compilation toolchain for high-performance and scalable quantum circuit simulation named CAST (Cross-platform Adaptive Schrödinger-style Simulation Toolchain). CAST introduces a custom intermediate layer and a cost-model based adaptive gate fusion algorithm. The algorithm automatically selects the best fusion configuration according to the characteristics of targeted CPU and GPU platforms. CAST incorporates high-performance kernel generators that automate backend support by emitting vectorized LLVM IR codes for a versatile SIMD solution on CPU platforms and PTX codes for utilizing CUDA cores.

Overall, this work makes the following contributions:

- **A novel adaptive gate fusion technique:** This technique considers the sparsity patterns and categories with an agglomerative scheme to apply gate fusion. We also adopt a cost-model based optimization that automatically selects fusion configuration based on targeted hardware. As a result, our tool can apply CPU or GPU-specialized gate fusion optimization to best match the characteristics of the target hardware.
- **A dynamic kernel generation approach:** CAST enables customized optimization to reduce the number of operations needed to perform quantum circuit simulation. Together with the sparsity-aware fusion technique, CAST achieves fast and energy-efficient quantum circuit simulation.
- **A cross-platform quantum simulation framework:** CAST serves as a cross-platform automated compilation toolchain for high-performance quantum simulation on CPU and Nvidia GPU platforms, featuring adaptive front-end optimization algorithms and high-performance backend support.

II. BACKGROUND AND MOTIVATION

Statevector-based Simulation Statevector-based simulation keeps track of all *amplitudes* in the statevector. An amplitude is a complex number, and an n -qubit statevector is a length- 2^n complex vector with norm 1. Let ψ be the length- 2^n complex vector that represents an n -qubit statevector. Applying a single-qubit gate U on qubit k is equivalently performing the matrix-vector multiplication specified by

$$(I^{\otimes(n-k-1)} \otimes U \otimes I^{\otimes k})\psi. \quad (1)$$

This expression represents multiplying a $2^n \times 2^n$ matrix with a $2^n \times 1$ vector. As the matrix is highly sparse, we can equivalently perform

TABLE I
COMPARISON OF VARIOUS CLASSICAL QUANTUM CIRCUIT SIMULATORS.

	Precision	CPU Backend SIMD Support ^a	CUDA Support	Gate Fusion			Dynamic Kernel
				Size	Sparsity	Parametrized	
QuEST [13]	f32 and f64	✗	Builtin	✗	✗	✗	✗
Qibo [14]	f32 and f64	External	CuPy	✓	Partial ^b	Limited	✗
Qulacs [15]	f32 and f64	f64 AVX2 only	Builtin	✓	✗	✓	✗
Qiskit [10]	f32 and f64	f64 AVX2 only	cuQuantum	✓	Partial ^c	✗	✗
QPanda [16]	f64 only	f64 AVX2 only	cuQuantum	✓	✗	Limited	✗
Cirq [11] ^d	f32 and f64	External	✗	✗	✗	✗	✗
QSimCirq [11]	f32 only	f32 AVX2 and f32 AVX512	cuQuantum	Up to 6-qubit	✗	Limited	✗
CUDA Quantum [17]	f32 and f64	External	cuQuantum	Up to 6-qubit	✗	Limited	✗
CAST (ours)	f32 and f64	✓ ^e	Builtin	✓	✓	✓	✓

^a“External” means this backend depends on external pre-compiled library, such as numpy’s C++ backend.

^bQibo provides qibojit as a just-in-time engine.

^cQiskit’s gate fusion algorithm does not consider general gate matrix sparsity, but it separates diagonal gates and general unitary gates.

^dCirq and QSimCirq (sometimes QSim) are both developed by Google. While Cirq has its own simulation backend, QSimCirq provides a high-performance backend specifically optimized for SSE, AVX2, and AVX512-capable hardware for simulating Cirq circuits.

^eTested to support NEON, SSE, AVX2, and AVX512, both f32 and f64 precision.

2^{n-1} matrix-vector multiplications in sequential, each time between U and two of the amplitudes of ψ to improve the efficiency. That is,

$$U \begin{bmatrix} \psi_{\alpha(t)} \\ \psi_{\beta(t)} \end{bmatrix}, \quad t = 0, \dots, 2^{n-1} - 1. \quad (2)$$

More generally, if U is a k -qubit gate represented by a $2^k \times 2^k$ unitary matrix, then applying U on an n -qubit statevector ψ consists of 2^{n-k} matrix-vector multiplications

$$U \begin{bmatrix} \psi_{\alpha_0(t)} \\ \vdots \\ \psi_{\alpha_{2^k-1}(t)} \end{bmatrix}, \quad t = 0, \dots, 2^{n-k} - 1. \quad (3)$$

Ahead-of-time Kernel Preparation While a dense single-qubit gate can have up to 8 non-zero scalars, each Pauli gate only has two. Simulators with support to only general quantum gates inevitably lose optimizations for Pauli gates. Therefore, for better performance, a traditional approach is to hand-code popular gate kernels ahead of time. Some simulators such as QSimCirq even write vector intrinsics to gain more performance on selected CPU platforms.

However, such a kernel preparation scheme induces extreme workload in writing different kernels to support every gate. With gate fusion, the workload is even greater. Fused gates usually act on more than two qubits, so their sparsity patterns are much more diverse and unpredictable. The pattern of loading and storing vector amplitudes also grows exponentially with respect to the gate size. To reduce workload, some simulators supporting gate fusion, such as Qulacs, QPanda, and QSimCirq, invoke general multi-qubit kernel on fused gates. Some other ones, such as Qiskit, separate diagonal matrices by providing additional diagonal multi-qubit gate kernels. However, none of them could fully appreciate the sparsity patterns of fused gates.

Motivation To facilitate the quantum simulation on classical computers, various quantum circuit simulators have been developed by both industry and academia [13], [14], [15], [10], [16], [11]. However, as summarized in Table I, most of these tools overlook the sparsity information during simulation and have limited backend support. As illustrated in Fig. 1, to leverage the sparsity and dynamic optimization opportunities, our simulator proposes a novel sparsity-aware gate fusion technique and a dynamic kernel generation process

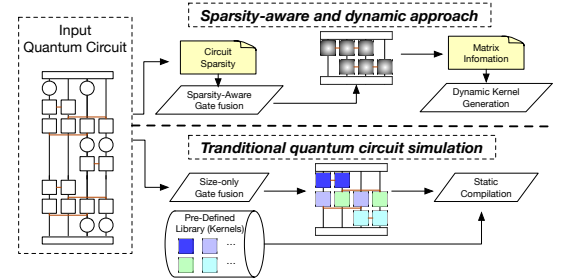


Fig. 1. The proposed CAST approach exploits sparsity information and runtime-matrix information to improve simulation performance.

that utilizes runtime matrix information for further optimizations. Compared with prior quantum circuit simulators, our tool enables more effective gate fusion with highly optimized kernels, significantly improving the simulation performance. By integrating these optimizations and backend solutions, our proposed toolchain delivers cross-platform, scalable, and high-performance quantum simulations.

III. OVERVIEW

A. CAST Compilation

CAST compilation consists of two main phases: an adaptive gate fuser and partitioner that transpiles input circuits via a custom data structure *CircuitTile*, and a dynamic kernel generator that emits and optionally compiles hardware-specific instructions. A schematic diagram is given in Fig. 2.

CAST’s workflow brings the following advantages:

- When targeting CPU based platforms, CAST provides versatile SIMD capabilities via LLVM IR backend. This avoids the traditional ahead-of-time approach by hard-coding multiple vector intrinsics for different SIMD instruction sets and simulation precisions.
- CAST optimizes sparse gates during kernel generation. For example, all zero entries in the matrix are skipped in generated kernels. This brings a cross-platform solution for the previously infeasible task of preparing optimized kernels for fused gates.
- CAST kernel generator is highly customizable and extendable. Kernel generators allow designers to obtain a consistent performance across platforms without repetitive work of implementing

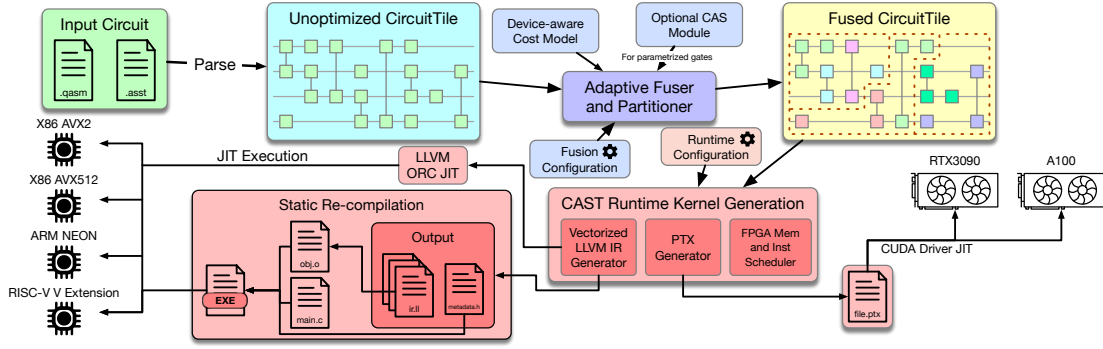


Fig. 2. CAST Workflow.

hardware-specific optimizations. Along the automation process, CAST also provides a lot of room for exploration as shown in Table II.

TABLE II
PART OF CAST SOFTWARE ARGUMENTS.

Argument	Command line
Precision	-f32, -f64
CPU	-S=<int>, -neon, -sse
vector size	-avx2, -avx512
Gate fusion	-cost-model=<cost-model>, -fusion=<int> -zero-tolerance=<float> -one-tolerance=<float> -multi-traversal=<bool> -agglomerative=<bool>
Kernel configurations	-arch=<cpu/gpu> -use-fma, -use-fms, -use-pdep -alt-format, -sep-format -zero-tolerance-kernel=<float> -elem-sharing-threshold=<float> -use-imm-value=<bool> -use-const-mem-space=<bool> -force-dense-kernel=<bool>

IV. ADAPTIVE SPARSITY-AWARE AGGLOMERATIVE GATE FUSION

A. CircuitTile Data Structure

Because the difficulty of gate fusion lies in preserving the commutation relation of quantum circuits, we design a custom data structure *CircuitTile* as an intermediate layer to represent quantum circuits and to perform gate fusion.

Internally, *CircuitTile* contains a linked list of arrays that we call *tile*. Each array in the tile is called a *row*. Every row is an array of null-able *GateBlock* pointers. A *GateBlock* is an aggregate of gates. Each gate keeps track of its left and right neighbors.

To add a gate to the graph, we create a *GateBlock* consisting of solely this gate and append it to the tile. Tile spans vertically by convention, so to append a *GateBlock* to the tile, we insert it to the uppermost vacant row.

B. Agglomerative Gate Fusion

Gates with disjoint target qubit sets always commute with each other. So in the tile, blocks on the same row can be safely fused together. On the other hand, gates not on the same row can be fused together only when they are connected and on consecutive rows.

In other words, a pair of gate blocks can be fused in two scenarios. The first scenario is **Commuting Gate Fusion**, where the two blocks

are on the same row. The second scenario is **Consecutive Gate Fusion**, where the two blocks are located in consecutive rows, and there is at least one direct connection spanning between them.

CAST employs tile-based iteration to perform gate fusion. The pseudo-code is provided in Algorithm 1. This algorithm takes in a *CircuitTile* T and an unsigned integer k as the input. It traverses through all gate blocks and fuse the blocks. During the execution, the number of qubits of the fused gate should stay below k to avoid fragments. The algorithm returns **true** if and only if there is at least one pair of gate block merged during the execution.

Algorithm 1 Tile Traversal in Agglomerative Gate Fusion

```

1: function TRAVERSE(CircuitTile  $T$ , unsigned  $k$ )
2:    $\delta \leftarrow \text{false}$ 
3:   for  $r \in [0..N_{\text{rows}} - 1]$  do
4:     for  $q \in [0..N_{\text{qubits}} - 1]$  do
5:        $b_{\top} \leftarrow T_{r,q}$ 
6:       if  $b_{\top} \neq \emptyset$  then
7:          $b_{\perp} \leftarrow T_{r+1,q}$ 
8:         if  $b_{\perp} \neq \emptyset$  then
9:           MOVEBLOCK( $b_{\top}, b_{\perp}$ )
10:          if FUSIBLE( $b_{\top}, b_{\perp}, k$ ) then
11:            FUSE( $b_{\top}, b_{\perp}$ ) ▷ Consecutive Fusion
12:             $\delta \leftarrow \text{true}$ 
13:          for  $q \in [1..N_{\text{qubits}} - 1]$  do
14:             $b_{\top} \leftarrow T_{r,q-1}$ 
15:             $b_{\perp} \leftarrow T_{r,q}$ 
16:            if FUSIBLE( $b_{\top}, b_{\perp}, k$ ) then ▷ Commuting Fusion
17:              FUSE( $b_{\top}, b_{\perp}$ )
18:               $\delta \leftarrow \text{true}$ 
19:   return  $\delta$ 

```

The algorithm actively relocates blocks to create fusion opportunities. Specifically, when the traversal reaches a gate block in the i -th row and the space below it in the $(i+1)$ -th row is unoccupied, the algorithm moves the gate block down to the $(i+1)$ -th row to fill the space. This relocation allows the block to potentially fuse with a block in the $(i+2)$ -th row. For example, Fig. 3 gives an illustration of such a case. In the compact tile above, blocks 1 and 3 are not in consecutive rows. Since the position immediately below block 1 is empty (line 8 in Algorithm 1), block 1 can be moved down. Block 1 will then fuse with block 3 when we traverse to the new position of block 1.

To place fused blocks back into the tile, the FUSE procedure first removes $b_{\top}$ and $b_{\perp}$. If the removal creates enough space in rows r or $(r+1)$ to accommodate the fused block, the fused gate block occupies the corresponding space (prioritize on row $(r+1)$). Otherwise, the FUSE procedure inserts a new row between the two rows and places the fused block in the new row. Note that the creation of the new

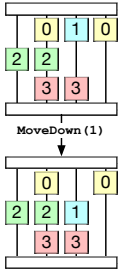


Fig. 3. MoveDown.

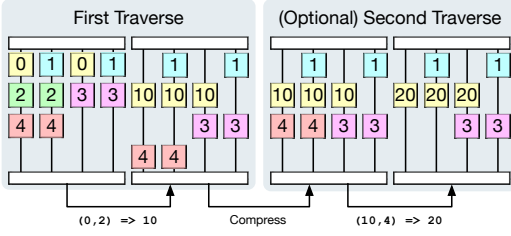


Fig. 4. Tile traversal.

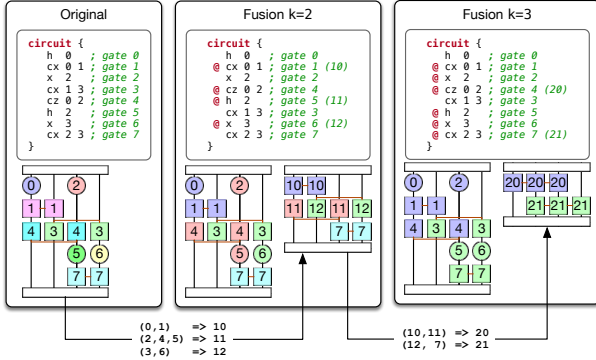


Fig. 5. Demonstration of agglomerative gate fusion. Each top block shows CAST serialization of the `CircuitTile`. The @ symbol denotes fusing gates together.

row can leave empty spaces (Fig. 4). To maintain the compactness of the `CircuitTile`, we compress it at the end of each tile traversal.

Given a specified maximum size $k_{\max}$ of fused gates, the agglomerative scheme iteratively increases the allowed size of fused gates from 2 to $k_{\max}$. We found that such an incrementing scheme helps reduce fragments (small gates remaining after the algorithm finishes) and leaves with overall sparser gates.

C. Adaptive Fusibility Check

Adaptive gate fusion traverses the tile the same way as before, but adds extra criteria in fusion eligibility checks. For CPU and GPU based adaptive gate fusion, we define a cost model upon the total operation count and the number of blocks present in the fused circuit.

The *operation count* captures the sparsity of gates, and is defined as the number of run-time operations needed to perform matrix-vector multiplication. Take single-qubit gates as an example, the matrix-vector multiplication (2) corresponds to updating four variables (registers) in each iteration. For machines with support of both Fused Multiplication Addition (FMA) and Fused Multiplication Subtraction (FMS), every non-zero entry in the matrix requires 2 operations in run-time. A dense k -qubit gate has 2^{2k+1} entries, and thus its operation count is 2^{2k+2} . On the other hand, the operation count for sparse gates such as Pauli gates is far less than the worst case described above.

Fig. 6 shows one example. For a fixed number of threads, there is a peak performance point when we vary gate fusion aggressiveness. Left to the peak point is usually when memory bottlenecks speed, and right to the peak point is usually when computation power bottlenecks speed.

CAST provides several preset fusion configurations for CPU and GPU based platforms. Users are also encouraged to use the our provided `CostModel` class to conduct benchmarks for a cost model

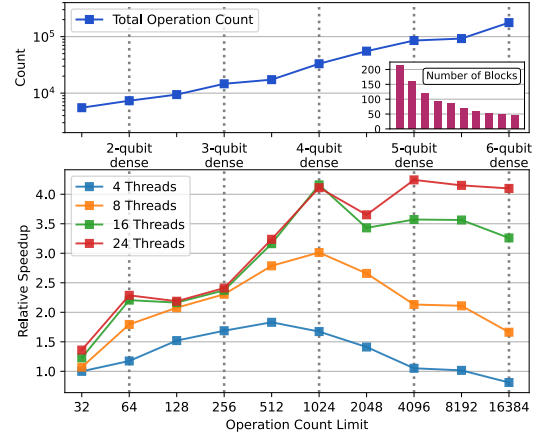


Fig. 6. Operation Count versus Simulation Time. Reference time is measured at the 4-thread 32-op-count-limit experiment. All experiments were carried out on CPU-based 30-qubit statevectors with double precision.

specialized to their hardware platform. `CostModel` will measure and record the running time of several quantum gates with varying sparsity and size and using different number of threads (for CPUs) and block dimensions (for Nvidia GPUs). These data will then be used in the fusion algorithm for adaptive fusibility checks. For example, on one of our CPU platforms, `CostModel` finds 6-qubit dense gates take significantly longer time to simulate than 5-qubit dense gates. So it decides to limit the operation count per gate to 4096 (equaling to that of a 5-qubit dense gate) while allowing fused gates to act on up to 7 qubits. As shown in Table III, in a sample Random Quantum Circuit (RQC), the adaptive configuration achieves a comparable compression ratio with a significantly reduced operation count.

TABLE III
FUSION CONFIGURATION COMPARISON.

Configuration	Compression Ratio	Fusion Time (ms)	Operation Count
Adaptive	22.1x	1423	1.05M
Size-only	23.2x	988	1.74M
Non-agglomerative	23.2x	6660	3.28M

V. DYNAMIC KERNEL GENERATION AND COMPILATION

A. Automated Kernel Generation

CAST automates the kernel generation process for gates with various sizes and target qubits. Each CAST CPU kernel takes 4 arguments and each GPU kernel takes 2 arguments. These arguments are summarized in the following list:

- 1) Pointer to the statevector array.
- 2) Pointer to the matrix array. This is to support simulating parametrized quantum circuits.
- 3) Loop counter begin index, present only in CPU kernels.
- 4) Loop counter end index, present only in CPU kernels.

CAST backend kernel generator starts by emitting LLVM IR. In CPU kernels, the loop counter runs in $[0, 2^{n-k-s})$, where n is the number of qubits, k is the gate size, and $S = 2^s$ specifies vector size to be used in CPU-based SIMD instructions. This range can then be distributed to different threads for parallel execution. GPU kernels do not take loop counter as arguments because the loop counter is effectively tracked via built-in `blockIdx` and `threadIdx` variables.

Each GPU thread has an effective vector size of 1. So the following IR generation process also works for GPUs with $s = 0$.

For a k -qubit gate, we need to update 2^{k+1} vector amplitude registers in the main loop. The address and contiguity pattern of the amplitudes heavily depend on SIMD size and the set of target qubits.

Loading amplitude registers has two sub-steps: first calculate the start index from loop counter t , then depending on amplitude format, load 2^k real and imaginary amplitude registers each. We first split target qubits into two sets, the higher set and the lower set. Start by enumerating indices from 0. Color target qubit indices blue and color the remaining s smallest numbers red. Everything smaller than or equal to the largest red index is on the lower side (as shown in Fig. 7).

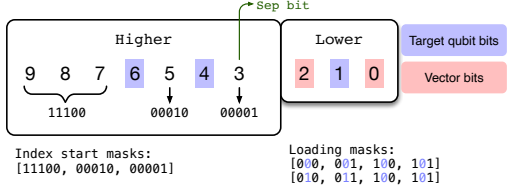


Fig. 7. Illustration when $s = 2$ and target qubits are $\{1, 4, 6\}$.

Let Q be the set of target qubits, H be the set of all higher indices, and L be the set of all lower indices. For some convenient numbers, let $k = |Q|$ be the size of the gate, and let $k_L = |Q \cap L|$ and $k_H = |Q \cap H|$ be the number of target qubits inside lower and higher set respectively. Then

- If $k_L = 0$, no runtime shuffling is needed both in loading and storing.
- Otherwise, the loop body loads (and stores) a total of 2^{k_H+1} length- 2^{k_L+s} LLVM vectors. These vectors will then be shuffled to 2^{k+1} length- 2^s vectors before matrix-vector multiplication. So on the LLVM IR level, when k_L is non-zero, shuffled loading works by loading larger LLVM registers and let the assembler split these registers and assign hardware-native shuffling instructions.
- Depending on hardware support, at most $3k_H + 1$ operations are needed to compute the start index:

$$\text{startIndex} = \sum_{i=0}^{|H \cap Q|} (t \& \text{masks}[i]) \ll i, \quad (4)$$

where masks is a compile-time known array¹.

B. JIT Execution and Re-compilation

As part of design flexibilities, CAST provides two modes for simulation on CPU-based platforms: JIT mode and static mode. JIT mode adopts LLVM ORC JIT engine to JIT-compile generated IR to native machine code for immediate execution. The static mode separate kernel generation and execution. It caches and saves generated IR files along with a header file of metadata. The saved IR files and metadata can then experience a compilation and linking phase to generate the final executable when needed.

GPU-based platforms follow a similar approach. The difference is that CUDA execution is internally JIT. CAST provides the flexibility of choosing to load the generated PTX files into a CUDA context in runtime or saving it to the disk for future needs.

¹For machines that support BMI2 instruction set, a single runtime PDEP operation can replace Eq. (4). This is automatically configured in CAST argument `-use-pdep`.

VI. EVALUATION

A. Setup

Software In our evaluations we used the following software:

- LLVM project: version 19.1.0.
- Qiskit: version 1.2.4 with backend QiskitAer 0.15.1.
- Qibo: version 0.2.13; QiboJIT: version 0.1.7.
- Cirq: version 1.4.1; QSimCirq: version 0.21.0.
- Qulacs: version 0.6.11.

Benchmark Circuits We used six types of benchmark circuits in our evaluations.

- Quantum Fourier Transform (QFT) [18].
- Alternating Layered Ansatz (ALA) [19].
- Random Quantum Circuit (RQC) [3].
- Quantum Volume Circuit (QVC) [20].
- Instantaneous Quantum Polynomial (IQP) [6].
- Hamiltonian Evolution Simulation (HES) [21].

Among the six types of circuits, QFT, RQC, IQP, and HES are relatively more sparse. So we call them *sparse-class* benchmarks. Correspondingly, ALA and QVC consist of mostly dense single-qubit gates, so we call them *dense-class* benchmarks.

B. Ablation Study on Adaptive Gate Fusion

To evaluate the performance improvement brought by our adaptive fusion algorithm, we perform an ablation study on different benchmark circuits with three fusion configurations: no fusion, preset size-only fusion, and adaptive fusion. We record the running time of each experiment under each fusion configuration and normalize it by the running time taken under the no fusion configuration.

As shown in Fig. 8, compared with the original circuit, size-only fusion optimization can reduce the running time by 68.7% ~ 93.3% (resp. 62.0% ~ 91.9%) over different benchmarks on the CPU (resp. GPU) platform. Compared with traditional size-only fusions, our new adaptive fusion algorithm obtains comparable performance on dense-class benchmarks, and can further reduce the running time by 28.7% ~ 40.4% (resp. 18.5% ~ 36.6%) on sparse-class benchmarks on the CPU (resp. GPU) platform.

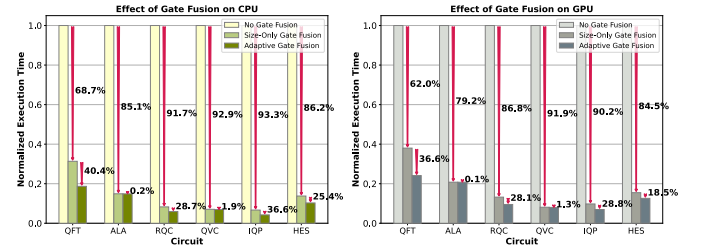


Fig. 8. Execution time of quantum simulation with different fusion strategies.

C. Cross-platform Backend

To demonstrate the effectiveness of CAST kernel generators on various platforms and various precisions, we benchmark and compare the performance of dynamic kernels generated by CAST and traditional ahead-of-time kernels as in Qiskit, Qibo, QSim, and Qulacs on the task of simulating gates with various sparsity on 28-qubit statevectors.

As reported in Table IV, our CAST backend achieves comparable performance on dense two-qubit and three-qubit gates compared with QSimCirq on AVX-512 platform `£32` mode and outperforms Qiskit, Qibo, QSim, and Qulacs on other benchmarks. This demonstrates our dynamically generated CAST backend has great versatility over different CPU and GPU platforms and simulation precisions. In addition, our backend generally runs faster on sparser gates, which

TABLE IV
COMPARISON OF SINGLE GATE PERFORMANCE.

Gate	Apple M2 (NEON)						Intel(R) Core(TM) i9-11900K (AVX512)						Nvidia RTX3090 (CUDA)					
	f 32 Mode (GiBps)			f 64 Mode (GiBps)			f 32 Mode (GiBps)			f 64 Mode (GiBps)			f 32 Mode (GiBps)			f 64 Mode (GiBps)		
	Qiskit	Qibo	Ours	Qiskit	Qibo	Ours	Qiskit	QSim	Ours	Qiskit	QSim	Ours	Qiskit	Qulacs	Ours	Qiskit	Qulacs	Ours
$H^{\otimes 2}$	1.74	3.20	32.9	3.46	6.04	34.1	1.28	15.6	17.3	2.50	N/A	17.1	75.2	N/A	416	181	150	417
$U3^{\otimes 2}$	1.74	3.18	20.7	3.45	5.87	21.1	1.28	15.5	16.5	2.62	N/A	16.2	75.4	N/A	418	180	151	320
$H^{\otimes 3}$	0.666	3.21	22.0	1.22	5.95	22.8	0.830	12.5	15.6	1.56	N/A	14.9	18.9	N/A	396	80.6	81.4	308
$U3^{\otimes 3}$	0.663	3.11	8.38	1.28	6.11	8.50	0.838	12.4	12.3	1.56	N/A	12.7	18.9	N/A	393	80.6	81.3	149

verifies CAST is able to automatically explore the sparsity pattern of gates.

D. Compilation Overhead

Compared with traditional approaches, CAST adopts extra code generation and re-compilation phases. We evaluate the impact of software overhead in this set of experiments.

We perform end-to-end simulations of RQCs with various number of qubits and record the normalized time spent in CAST front-end and hardware backend respectively. The front-end consists of the full process from parsing input files to obtaining executable. Normalized time is calculated by dividing the total end-to-end time spent with default fusion preset by the number of gates in the original circuit. As reported in Fig. 9, the ratio of CAST front-end overhead diminishes quickly as number of qubits increases. For example, in the 32-qubit experiment, the front-end of CAST only takes 0.59% (resp. 3.11%) of the end-to-end running time under the default (resp. adaptive) fusion configuration. This demonstrates CAST has great scalability in simulations. On the other hand, this set of experiments also suggests our adaptive fusion scheme exchanges slight front-end overhead for a great performance gain in running time.

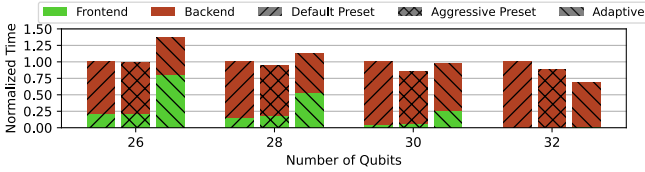


Fig. 9. Front-end overhead.

E. Full Circuit Simulation

We benchmark our CAST on full circuit simulations on CPU and GPU platforms.

CPU Performance We benchmark our simulator against Qiskit, Qibo, and QSimCirq on 32-qubit circuit simulations with multi-threading enabled. We record the execution time as reported by each simulator. Results are presented in Table V.

For single-precision mode simulations, compared with QSimCirq, our simulator achieves comparable performance on dense-class circuits and up to 3.68x speedup on sparse-class circuits such as QFT-30. This indicates our new sparsity-aware gate fusion algorithm is particularly effective in sparse circuits while still attaining a top performance on dense circuits. Over different types of benchmarks, CAST achieves 3.58x ~ 8.03x (resp. 2.14x ~ 4.92x) speedup than Qiskit and 3.60x ~ 17.4x (resp. 2.29x ~ 12.0x) speedup than Qibo in single-precision (resp. double-precision) mode simulations.

GPU Performance We benchmark our simulator against Qiskit and Qulacs on 30-qubit circuit simulations. We record the execution

TABLE V
COMPARISON OF CPU PERFORMANCE.

Circuit	AMD EPYC 7543 (32-core, AVX2)							
	Single Precision (s)				Double Precision (s)			
	Qiskit	Qibo	QSim	Ours	Qiskit	Qibo	QSim	Ours
QFT-32	145	427	98.8	27.4	190	574	N/A	51.4
ALA-32	876	1011	204	176	956	1342	N/A	337
RQC-32	190	237	71.1	26.8	205	304	N/A	55.8
QVC-32	409	430	110	113	467	572	N/A	223
IQP-32	589	327	82.1	25.3	1087	420	N/A	49.6
HES-32	1918	1867	605	201	3457	2176	N/A	408

TABLE VI
COMPARISON OF GPU PERFORMANCE.

Circuit	Nvidia RTX3090					
	Single Precision (s)			Double Precision (s)		
	Qiskit	Qulacs	Ours	Qiskit	Qulacs	Ours
QFT-30	16.4	N/A	2.44	12.0	52.4	5.44
ALA-30	131	N/A	9.83	99.2	96.6	28.3
RQC-30	307	N/A	15.9	220	283	79.7
QVC-30	198	N/A	5.04	161	53.1	25.5
IQP-30	41.0	N/A	2.01	45.2	52.4	5.08
HES-30	683	N/A	81.4	479	1260	173

time reported in each software. Qulacs has its own CUDA backend that only supports double-precision simulations. The GPU backend of Qiskit is cuQuantum [22], which is developed by Nvidia and supports both f32 and f64 format simulation (Table I).

As shown in Table VI, over different types of benchmarks, CAST achieves 6.71x ~ 39.3x speedup compared with Qiskit in single-precision mode simulations, and 2.08x ~ 10.3x speedup compared with Qiskit and Qulacs in double-precision mode simulations.

The results demonstrate our toolchain has great performance and versatility in both single and double-precision modes and on different hardware platforms.

VII. CONCLUSION

This paper proposes a high-performance cross-platform quantum circuit simulation framework CAST (Cross-platform Adaptive Schrödinger-style Simulation Toolchain). CAST automates circuit optimization and kernel generation process by introducing a novel adaptive gate fusion and optimization algorithm and cross-platform backend kernel generators. Extensive evaluations suggest CAST can achieve higher performance over the leading industrial and academic quantum circuit simulators on large-size quantum circuit simulations with a minimized compilation overhead.

VIII. ACKNOWLEDGMENT

The support of UK EPSRC (grant number EP/W032635/1, EP/V028251/1, EP/S030069/1 and EP/X036006/1), Intel and AMD is gratefully acknowledged.

REFERENCES

- [1] J. Tian, X. Sun, Y. Du, S. Zhao, Q. Liu, K. Zhang, W. Yi, W. Huang, C. Wang, X. Wu *et al.*, “Recent advances for quantum neural networks in generative learning,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 12 321–12 340, 2023.
- [2] K. Blekos, D. Brand, A. Ceschini, C.-H. Chou, R.-H. Li, K. Pandya, and A. Summer, “A review on quantum approximate optimization algorithm and its variants,” *Physics Reports*, vol. 1068, pp. 1–66, 2024.
- [3] S. Boixo, S. V. Isakov, V. N. Smelyanskiy, R. Babbush, N. Ding, Z. Jiang, M. J. Bremner, J. M. Martinis, and H. Neven, “Characterizing quantum supremacy in near-term devices,” *Nature Physics*, vol. 14, no. 6, pp. 595–600, 2018.
- [4] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.
- [5] Y. Liu, X. Liu, F. Li, H. Fu, Y. Yang, J. Song, P. Zhao, Z. Wang, D. Peng, H. Chen *et al.*, “Closing the “quantum supremacy” gap: achieving real-time simulation of a random quantum circuit using a new sunway supercomputer,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–12.
- [6] M. J. Bremner, A. Montanaro, and D. J. Shepherd, “Achieving quantum supremacy with sparse and noisy commuting quantum computations,” *Quantum*, vol. 1, p. 8, 2017.
- [7] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio *et al.*, “Variational quantum algorithms,” *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [8] N. Hatano and M. Suzuki, “Finding exponential product formulas of higher orders,” in *Quantum annealing and other optimization methods*. Springer, 2005, pp. 37–68.
- [9] S. Aaronson and L. Chen, “Complexity-theoretic foundations of quantum supremacy experiments,” *arXiv preprint arXiv:1612.05903*, 2016.
- [10] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, “Quantum computing with Qiskit,” 2024.
- [11] Q. A. team and collaborators, “qsim,” Sep. 2020. [Online]. Available: <https://doi.org/10.5281/zenodo.4023103>
- [12] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi *et al.*, “PennyLane: Automatic differentiation of hybrid quantum-classical computations,” *arXiv preprint arXiv:1811.04968*, 2018.
- [13] T. Jones, A. Brown, I. Bush, and S. C. Benjamin, “Quest and high performance simulation of quantum computers,” *Scientific reports*, vol. 9, no. 1, p. 10736, 2019.
- [14] S. Efthymiou, S. Ramos-Calderer, C. Bravo-Prieto, A. Pérez-Salinas, D. García-Martín, A. García-Saez, J. I. Latorre, and S. Carrazza, “Qibo: a framework for quantum simulation with hardware acceleration,” *Quantum Science and Technology*, vol. 7, no. 1, p. 015018, 2021.
- [15] Y. Suzuki, Y. Kawase, Y. Masumura, Y. Hiraga, M. Nakadai, J. Chen, K. M. Nakanishi, K. Mitarai, R. Imai, S. Tamiya *et al.*, “Qulacs: a fast and versatile quantum circuit simulator for research purpose,” *Quantum*, vol. 5, p. 559, 2021.
- [16] M. Dou, T. Zou, Y. Fang, J. Wang, D. Zhao, L. Yu, B. Chen, W. Guo, Y. Li, Z. Chen *et al.*, “Qpanda: high-performance quantum computing framework for multiple application scenarios,” *arXiv preprint arXiv:2212.14201*, 2022.
- [17] J.-S. Kim, A. McCaskey, B. Heim, M. Modani, S. Stanwyck, and T. Costa, “Cuda quantum: The platform for integrated quantum-classical computing,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–4.
- [18] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 124–134.
- [19] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles, “Cost function dependent barren plateaus in shallow parametrized quantum circuits,” *Nature communications*, vol. 12, no. 1, p. 1791, 2021.
- [20] A. W. Cross, L. S. Bishop, S. Sheldon, P. D. Nation, and J. M. Gambetta, “Validating quantum computers using randomized model circuits,” *Physical Review A*, vol. 100, no. 3, p. 032328, 2019.
- [21] G. H. Low and I. L. Chuang, “Optimal hamiltonian simulation by quantum signal processing,” *Physical review letters*, vol. 118, no. 1, p. 010501, 2017.
- [22] J. Faj, I. Peng, J. Wahlgren, and S. Markidis, “Quantum computer simulations at warp speed: Assessing the impact of gpu acceleration,” *arXiv preprint arXiv:2307.14860*, 2023.